Università degli Studi di Milano Bicocca

Dipartimento di Informatica

Corso di Laurea Magistrale



Progetto di ML – Letter Recognition

ANNO ACCADEMICO 2024/2025

Barbera Matteo - 887512

Pellegatta Luca – 885966

Villa Emma – 885906

Indice

1. Introduzione 3
   1.1 Descrizione del Dominio 3
   1.2 Obiettivi 3
2. Dataset 4
   2.1 Struttura del Dataset 4
   2.2 Analisi Esplorativa 5
   2.3 PCA 7
3. Modelli 8
   3.1 Reti Neurali 9
   3.2 SVM 11
4. Metodi di validazione 12
   4.1 K-Fold Cross Validation su Reti Neurali 12
   4.2 GridSearch Cross Validation su Support Vector Machine 13
5. Analisi dei Risultati 15
   5.1 Reti Neurali 16
   5.2 SVM 19
6. Conclusioni 21

1. Introduzione

1.1 Descrizione del Dominio

Il problema trattato in questo progetto di gruppo riguarda la classificazione automatica delle lettere maiuscole appartenenti all'alfabeto latino scritte a mano.

Il task che ci prefiggiamo di risolvere ha una rilevanza significativa, poiché ha parecchie implicazioni pratiche, che spaziano dalla digitalizzazione di documenti alla trascrizione automatica di appunti scritti a mano. Pertanto, lo sviluppo di modelli di machine learning in questo contesto può avere un impatto concreto in molteplici settori.

1.2 Obiettivi

Il nostro obiettivo è quello di indurre due modelli di machine learning in grado di classificare correttamente gran parte delle istanze presentate, mettendoli successivamente a confronto per stabilire quale sia il migliore. L'approccio adottato è metodico e critico, analizzando ogni scelta operata, dalla selezione dei modelli utilizzati alla valutazione delle metriche degli stessi passando per una selezione dei migliori iperparametri per ciascuno di essi.
Infine, metteremo a confronto i risultati ottenuti per stabilire quale dei due approcci è quello migliore in termini prestazionali.

Per garantire una valutazione delle prestazioni dei modelli, verranno prodotte delle statistiche, come l'intervallo di confidenza della t-student, che ci permettano di affermare con elevata confidenza statistica l'accuratezza raggiunta dai modelli e la bontà degli stessi.

2. Dataset

Il dataset è stato recuperato dal repository della UC Irvine, recuperabile a questo link. Contiene feature numeriche dei campioni di immagini delle lettere maiuscole latine scritte a mano in riquadro bianco e nero, distorcendo in modo randomico, di campione in campione, 20 font diversi, al fine di produrre le diverse istanze che popolano il dataset.

Si presenta come un file .CSV che abbiamo importato con la libreria Pandas.

2.1 Struttura del Dataset

Il dataset presenta 20.000 esempi, caratterizzati da 16 feature più la colonna della label. Il valore di ogni feature è stato normalizzato all'origine, in un intervallo di interi di [0, 15]. La colonna target è categorica, corrisponde alla lettera caratterizzante (A, B, ..). La semantica del dataset è la seguente:

1. **x-box**: posizione orizzontale del bounding box (colonna minima)
2. **y-box**: posizione verticale del bounding box (riga minima)
3. **width**: larghezza del bounding box
4. **high**: altezza del bounding box
5. **onpix**: numero totale di pixel "accesi" (cioè della lettera)
6. **x-bar**: media delle coordinate x dei pixel "accesi"
7. **y-bar**: media delle coordinate y dei pixel "accesi"
8. **x2bar**: varianza delle coordinate x
9. **y2bar**: varianza delle coordinate y
10. **xybar**: covarianza tra x e y
11. **x2ybr**: media di $x^2 * y$
12. **xy2br**: media di $x * y^2$
13. **x-ege**: edge count medio (transizioni) lungo le righe nel ROI della lettera

14. **xegvy**: correlazione (coefficiente di Pearson) tra l'indice di riga e il conteggio delle transizioni orizzontali
15. **y-ege**: edge count medio lungo le colonne nel ROI
16. **yegvx**: correlazione tra l'indice di colonna e il conteggio delle transizioni verticali

2.2 Analisi Esplorativa

In questo paragrafo eseguiremo un'analisi esplorativa per verificare la qualità del dataset, concentrandoci particolarmente sulla popolosità di ciascuna classe e sull'identificare le feature che risultano avere una maggiore importanza nel riconoscimento dei singoli caratteri.

Osservando la popolosità delle classi si è riscontrato che quella del carattere "U" ha la dimensione maggiore con 813 elementi mentre le classi dei caratteri "H" e "Z" risultano essere quelle di dimensione minore aventi 734 elementi l'una.

La differenza tra la classe più popolosa e le classi meno popolose risulta essere di 79 elementi, pari allo 0.4% dell'intero dataset; ciò dimostra che il dataset è molto bilanciato e non sono presenti classi eccessivamente sovrarappresentate o sottorappresentate (come mostrato anche in Fig.1).

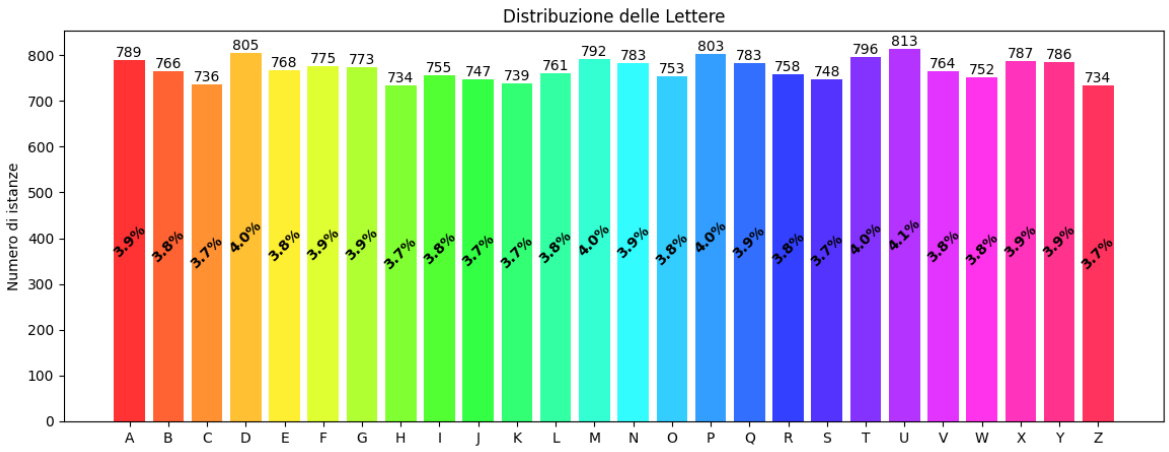


Fig.1: distribuzione dei vari caratteri all'interno del dataset

Per valutare l'impatto di ciascuna feature abbiamo calcolato alcune delle statistiche descrittive aventi ciascuna feature decidendo di tralasciare i valori massimi, minimi e i

quartili perché essendo i dati normalizzati tra 0 e 15 risultavano essere poco significativi per una analisi.

Quelle invece ritenute significative sono la mediana, la media e la deviazione standard; i valori riscontrati per ciascuna feature sono riportati nella Tab.1.

Analizzando le deviazioni standard, abbiamo riscontrato che la feature avente il valore maggiore è **y-box** con un 3,30 mentre le feature **xegevy** e **yegevx** risultano quelle con i valori minori, rispettivamente di 1,54 e 1,61; considerando che tutti i dati sono normalizzati possiamo dedurre che le feature con deviazioni più ampie siano anche quelle più significative da tenere in considerazione per la classificazione.

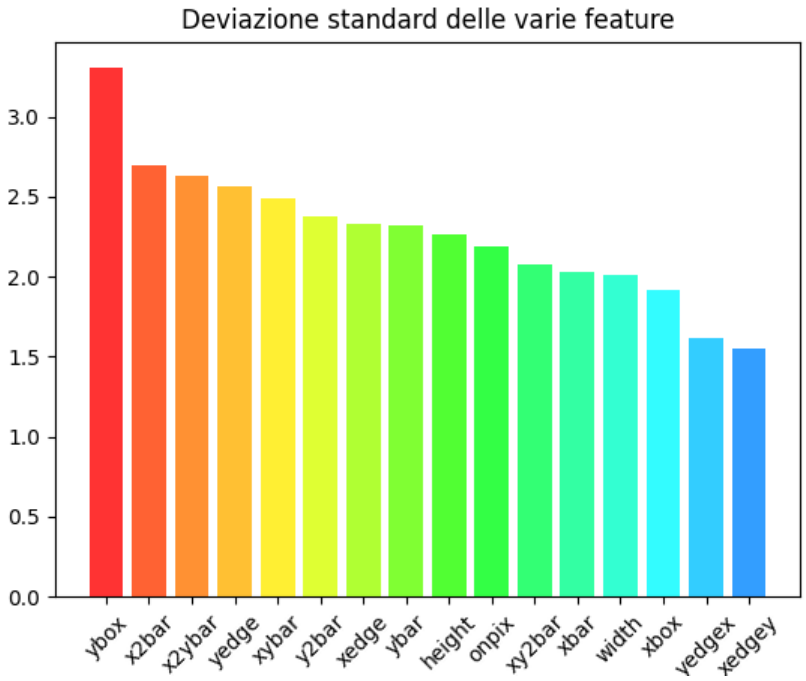


Fig.2: deviazioni standard aventi ciascuna feature,
ordinate partendo da quelle più significative per la classificazione

Di seguito è riportata una tabella che mostra alcune statistiche descrittive per ciascuna feature del dataset, ogni numero corrisponde a una feature, numerazione confrontabile nella lista fatta precedentemente. Sono stati riportati media, mediana e deviazione standard.

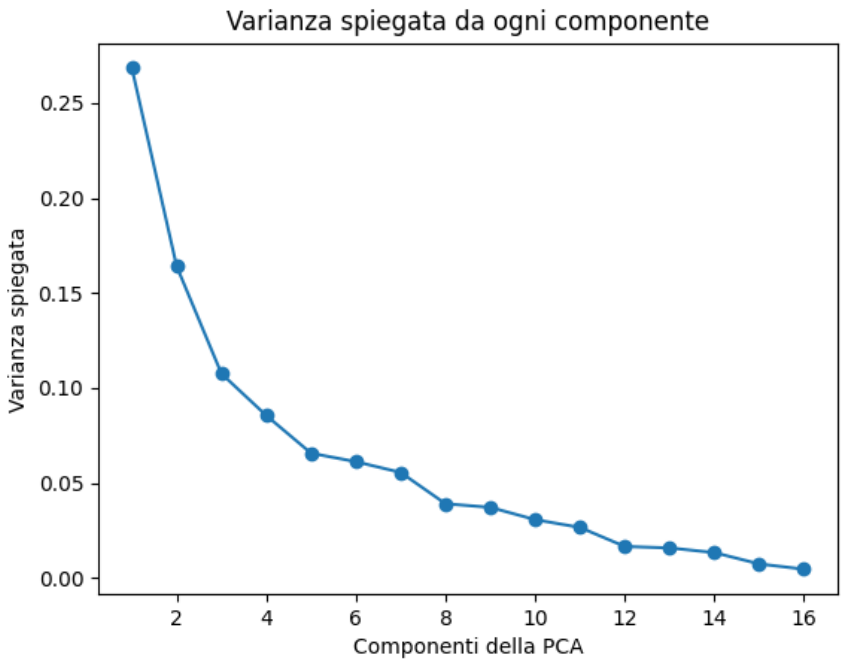
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
median	4	7	5	6	3	7	7	4	5	8	6	8	3	8	3	8
mean	4,02	7,03	5,12	5,37	3,50	6,89	7,50	4,62	5,17	8,28	6,45	7,92	3,04	8,33	3,69	7,80
std	1,91	3,30	2,01	2,26	2,19	2,02	2,32	2,69	2,38	2,48	2,63	2,08	2,33	1,54	2,56	1,61

Tab.1: statistiche descrittive di ciascuna feature

2.3 PCA

La Principal Component Analysis (PCA) ci consente di ridurre la dimensionalità (numero di feature) del dataset trasformando un insieme di feature fra loro correlate in un insieme di componenti scorrelate.

Prima di procedere, abbiamo prodotto un grafico che mostra la varianza spiegata dalle componenti principali.



Con le prime 6 componenti riusciamo a spiegare l'80% della varianza totale del dataset.

Tuttavia, abbiamo deciso di non applicare questa tecnica in quanto abbiamo ritenuto che applicandola saremmo andati incontro ad una perdita di informazione troppo significativa in relazione alla natura del problema.

3. Modelli

Fra i modelli visti a lezione, abbiamo scelto di utilizzarne due che seguono un approccio supervisionato, in quanto data l'eterogeneità del dataset e il numero relativamente alto di classi da cui è caratterizzato, non abbiamo ritenuto valido un approccio non supervisionato come k-means.

Data la scelta di utilizzare modelli di apprendimento supervisionati, il nostro obiettivo è anche quello di non andare in ***overfitting***, fenomeno dell'approccio supervisionato che si verifica quando il modello viene addestrato eccessivamente sui dati di training, specializzandosi in questi e perdendo la capacità di generalizzare sui dati nuovi.

A tal fine, verranno utilizzate tecniche di validazione descritte in seguito per evitare questo fenomeno, e garantire un bilancimento ottimale tra accuratezza e capacità di generalizzare.

Così come l'overfitting, cerchiamo di evitare l'***underfitting***, situazione duale che si verifica quando il modello è caratterizzato da una bassa accuratezza, ossia scarsa capacità di assegnare correttamente l'etichetta corretta alle varie istanze, sintomo di un addestramento insufficiente.

Abbiamo perciò scelto le **Reti neurali** e le **Support Vector Machine**, scartando l'approccio Naive Bayes in quanto, essendo le feature numeriche di un'immagine fortemente dipendenti fra loro, non sarebbe stato il più proficuo.

A tal proposito abbiamo effettuato un veloce esperimento empirico per verificare la correttezza della nostra ipotesi, addestrando un modello Naive Bayes, e ottenendo, secondo le nostre aspettative, una accuratezza di circa il **54%** (pessima, considerando che il modello sbaglia in quasi metà dei casi).

Abbiamo scartato anche gli alberi di decisione data la numerosità del dataset e la numerosità delle classi, oltre che il rischio di andare in overfitting, o in underfitting nel caso in cui avessimo deciso di limitare la profondità dell'albero.

In generale, abbiamo ritenuto che i modelli da noi scelti fossero più performanti in termini di accuracy, e più interessanti da un punto di vista tendenziale.

3.1 Reti Neurali

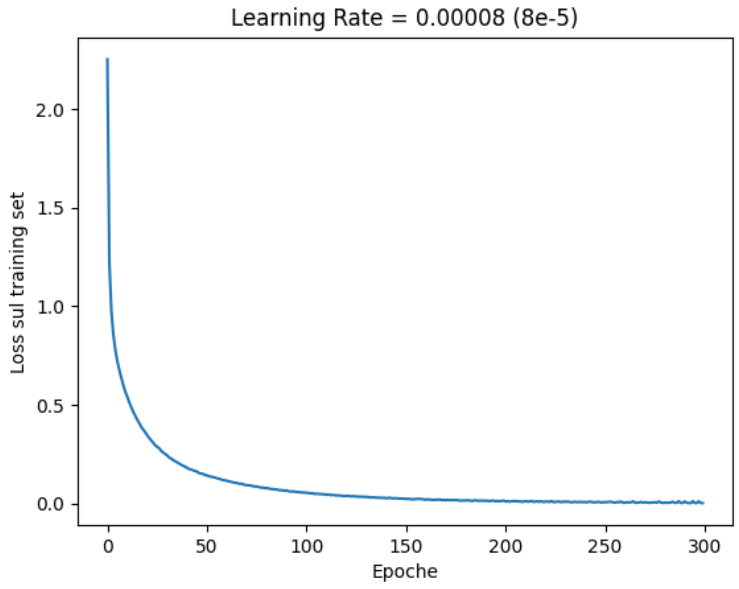
La nostra rete neurale è composta da 4 layer densi di dimensione (16, 256, 128, 26). Lo strato di input ha numerosità equivalente allo spazio delle feature, lo strato di output risulta essere di dimensione pari al numero di lettere maiuscole appartenenti al nostro alfabeto e infine le dimensioni degli strati nascosti sono state ottenute testando vari parametri (range 40-500). Abbiamo riscontrato che i valori 256 e 128 fornivano il miglior rapporto fra prestazioni computazionali e accuratezza finale del modello.

Le funzioni di attivazione nei vari strati sono ReLU e Softmax, abbiamo utilizzato la prima per gli strati di input e hidden mentre la seconda per lo strato di output.
La decisione di utilizzare Softmax è dovuta al tipo di classificazione richiesta dal problema quindi una classificazione multiclasse con singola uscita.

In fase di progettazione dell'architettura della rete è stata presa in considerazione l'idea di far variare il learning rate, parametro che stabilisce quanto velocemente il modello andrà poi a convergere, sostanzialmente "imparando" più velocemente quanto più alto è il parametro.

A tale scopo, abbiamo effettuato diverse prove per stabilire il learning rate ottimale, assicurandoci che non fosse troppo alto, impedendo così al modello di convergere per via del classico comportamento oscillante della loss, e neppure troppo basso da risultare irrilevante sull'aggiornamento dei pesi per ogni batch (size = 32).
Abbiamo deciso di utilizzare Adam, un algoritmo stocastico basato su Gradient Descent, con un learning rate di $8 \cdot 10^{-5}$.

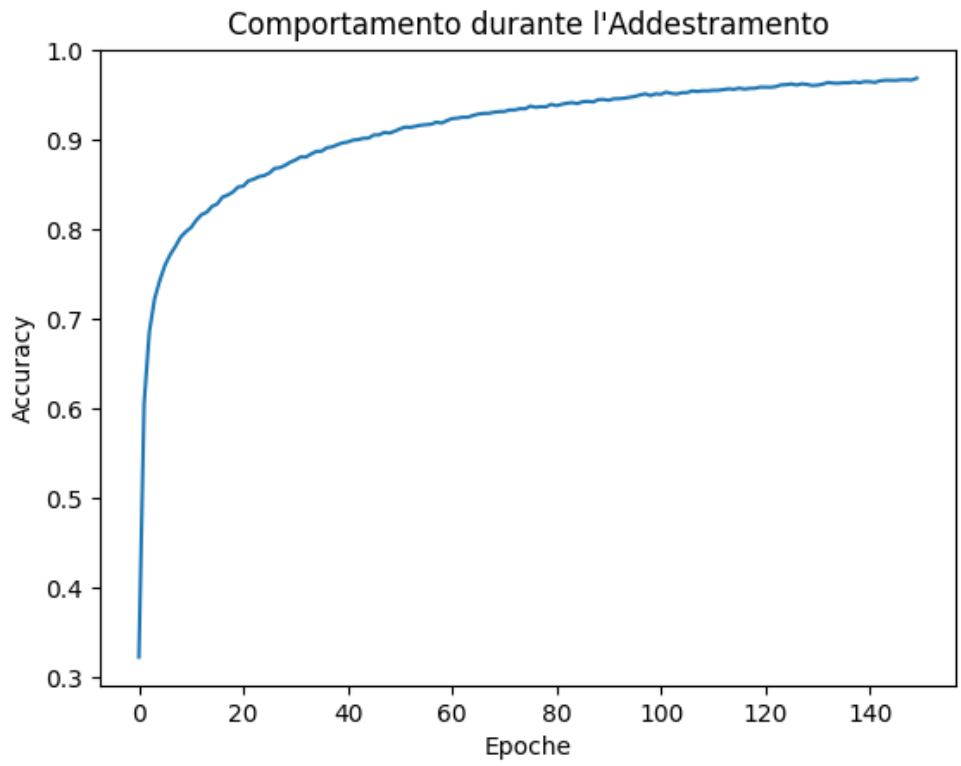
Con questo valore, riusciamo ad ottenere il seguente comportamento:



La relazione loss/epoche è abbastanza regolare e monotona, senza oscillazioni prima delle 200 epoche, motivo per cui abbiamo deciso di fissare a 150 il numero di epoche per cui addestrare la nostra rete.
Se avessimo impostato il numero delle epoche ad un valore più alto, saremmo andati incontro al fenomeno dell'overfitting.
La funzione di loss utilizzata è la categorical_crossentropy, ossia:

$$H(p,\, q) \;=\; \sum_{x \in c} p(x) \log q\,(x)$$

In cui p(x) è la distribuzione di probabilità della ground truth, mentre q(x) è la distribuzione predetta dal modello. Questa loss è definita nella libreria Keras ed è adattata per operare con problemi di classificazione multiclasse come il nostro.



Al termine dell'addestramento abbiamo effettuato un'analisi preliminare del modello sui dati di test, riuscendo così a determinare l'accuracy e la loss sulla partizione di dati che il modello non ha mai visto, ottenendo un'accuratezza del 95.17% e una loss del 15.81%.
Questo ci indica bontà del modello: infatti il modello classifica correttamente il 95% delle istanze e assegna una probabilità $p \;=\; e^{-0.15} \cong 0.86$ circa alla label corretta.

Questa analisi preliminare non ci dà garanzie sull'effettiva capacità del modello di generalizzare correttamente: non riusciamo a garantire l'assenza di overfitting, cosa a cui ovviamo utilizzando tecniche di validazione, descritte in seguito.

3.2 SVM

Al fine di addestrare la Support Vector Machine, la suddivisione del dataset in due partizioni utilizzata per le reti neurali è adattabile al contesto.
La libreria sklearn ci mette a disposizione più kernel, ossia funzioni che mappano uno spazio n-dimensionale in uno spazio m-dimensionale con $m > n$ al fine di separare meglio i vari esempi.

Tra le funzioni kernel disponibili, incontriamo:

- Linear: kernel lineare
- Polynomial: kernel polinomiale
- RBF: kernel con funzione radiale di base

La numerosità dello spazio delle feature ci suggerisce che l'utilizzo di un kernel non lineare sarebbe più appropriato, in quanto una separazione meramente lineare potrebbe non riuscire a discriminare bene le varie classi.
Abbiamo dunque deciso di addestrare una SVM "naïve" che utilizza un kernel lineare per verificare il valore di accuracy che il modello riesce ad ottenere sui dati di test.

Il valore riscontrato è dell'85%, lasciando invariato il valore di regolarizzazione (C = 1.0), ossia un parametro di costo che, più alto risulta, più rigorosa sarà la separazione, quindi più stretto sarà il margine e più alto sarà il rischio di overfitting.

Il valore di accuracy, comunque superiore alle aspettative, risulta inferiore a quanto ottenuto dalla rete neurale precedentemente addestrata; quindi, cerchiamo di migliorare le prestazioni variando i parametri e il kernel utilizzato.

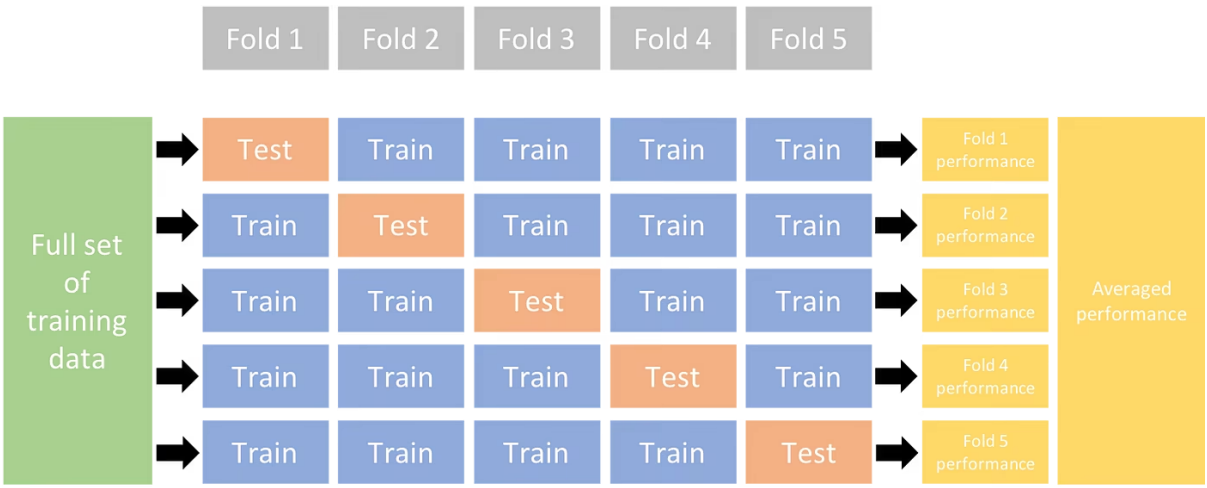
4. Metodi di validazione

Al fine di garantire capacità di generalizzazione del modello e quindi scongiurare il fenomeno dell'overfitting, è possibile utilizzare delle tecniche di validazione, ossia delle metodologie che seguendo un approccio iterativo suddividono l'intero dataset in training set, test set e validation set: quest'ultimo è una partizione del dataset che il modello utilizza, durante l'addestramento, insieme al training set, che ci avvisa, in caso di cattiva predizione su questi, di quanto sia proficua la scelta degli iperparametri per quel modello.

4.1 K-Fold Cross Validation su Reti Neurali

La tecnica K-Fold Cross Validation ci consente di suddividere il dataset in K partizioni di uguale dimensione, e a turno, una di queste costituirà il validation set.

Scegliendo un valore di **k = 5**, abbiamo ottenuto 5 partizioni di dimensione 4.000 ciascuna, in quanto il nostro dataset contiene 20.000 istanze.



Nel contesto delle reti neurali, abbiamo utilizzato questa tecnica con l'obiettivo finale di produrre un intervallo di confidenza che ci permetta di affermare con una certa confidenza, la differenza stimata tra un modello e l'altro ottenuto in fase di validazione, utilizzando partizioni diverse del dataset.

La libreria sklearn ci mette a disposizione un oggetto KFold che possiamo utilizzare per produrre una serie di indici del DataFrame, specificando k, che andranno a denotare

quali elementi marcare, ad ogni iterazione, come elementi di training o di validation.
A questo punto, in un ciclo, andiamo a riaddestrare la rete neurale, resettando ad ogni iterazione i pesi, per k volte, utilizzando stavolta la callback di EarlyStopping con valore di patience = 3, che ci permette di interrompere l'allenamento qualora il valore della loss riscontrato nelle ultime 3 epoche non dovesse più diminuire di un certo epsilon.
Al termine di ognuno dei k addestramenti, calcoliamo la accuracy ottenuta e la inseriamo all'interno di una lista.

Fuori dal ciclo, possiamo perciò produrre l'intervallo di confidenza mediante test della t-student con una confidenza del 90%, che risulta essere tra il 92% e il 94%.
Questo ci indica pragmaticamente che la rete neurale non va in overfitting, indicando bontà del modello di classificazione: ci aspettiamo dunque che la reale prestazione della rete neurale ricada in questo intervallo.

4.2 GridSearch Cross Validation su Support Vector Machine

Come già detto, una SVM può utilizzare una funzione kernel per migliorare la separazione.
Spesso e volentieri, l'utilizzo di un kernel specifico richiede la definizione di parametri aggiuntivi: è il caso del kernel RBF, che richiede la definizione di un parametro gamma, ossia l'inverso del raggio di influenza delle istanze scelte dal modello come vettori di supporto. Inoltre, come visto prima, la SVM richiede un parametro di regolarizzazione, ossia C.
La GridSearch Cross Validation ci permette di inferire i parametri migliori per C, gamma e il kernel stesso andando ad allenare più modelli, provando tutte le combinazioni possibili in dei range prefissati, sfruttando il principio di validazione.

Nel nostro caso, abbiamo definito 21 parametri possibili per C, secondo questa relazione:

$$C_i = C_{i-1} \cdot \frac{3}{2}$$

E impostando un valore iniziale di $C_0 = 0.1$.
Per quanto riguarda il valore di gamma, abbiamo impostato solo 3 valori: 0.1, 1, 10, mentre abbiamo provato a determinare il miglior kernel tra RBF e Poly, in quanto, come già sperimentato, il kernel lineare non sembra adatto per questo problema di classificazione.
Abbiamo infine scelto un valore di k = 5 per lo Stratified K-Fold, tecnica di validazione che GridSearchCV utilizza.

Il numero di addestramenti totali corrisponde quindi a:

$$n = |C| \cdot |gamma| \cdot |ker| \cdot k = 21 \cdot 3 \cdot 2 \cdot 5 = 630$$

La GridSearch, come noto e intuibile, è abbastanza dispendiosa dal punto di vista computazionale, in quanto deve provare tutte le combinazioni possibili di valori tenendo memoria anche delle prestazioni, al fine di valutare la combinazione migliore a fronte dei vari validation set.

Al termine dell'esecuzione, i valori ritenuti ottimali erano:

- **C**: 12.974
- **gamma:** 0.1
- **Kernel:** RBF

Valori grazie ai quali abbiamo ottenuto un'accuratezza della SVM di oltre il **97%,** valore, anche se di poco, superiore alla Rete Neurale.

5. Analisi dei Risultati

Per valutare le prestazioni e la qualità dei nostri modelli abbiamo confrontato diverse metriche e prodotto dei grafici, fra questi: le matrici di confusione, le curve ROC, gli indici di valutazione delle performance.

Nella **matrice di confusione**, gli elementi sulla diagonale rappresentano i veri positivi, ossia le istanze classificate correttamente; usando una classe cromatica più chiara per alte numerosità, risulta banale concludere che le istanze classificate non correttamente dal modello sono pochissime.

La **ROC Curve** consiste in una visualizzazione su piano cartesiano dell'area sottesa (AUC) ad ogni curva che esprime la relazione tra TPR e FPR (True Positive/False Positive Rate) relativa ad ogni classe. Per avere una rappresentazione chiara, avremmo dovuto plottare un numero di grafici pari al numero di classi, adottando una rappresentazione One-vs-Rest.

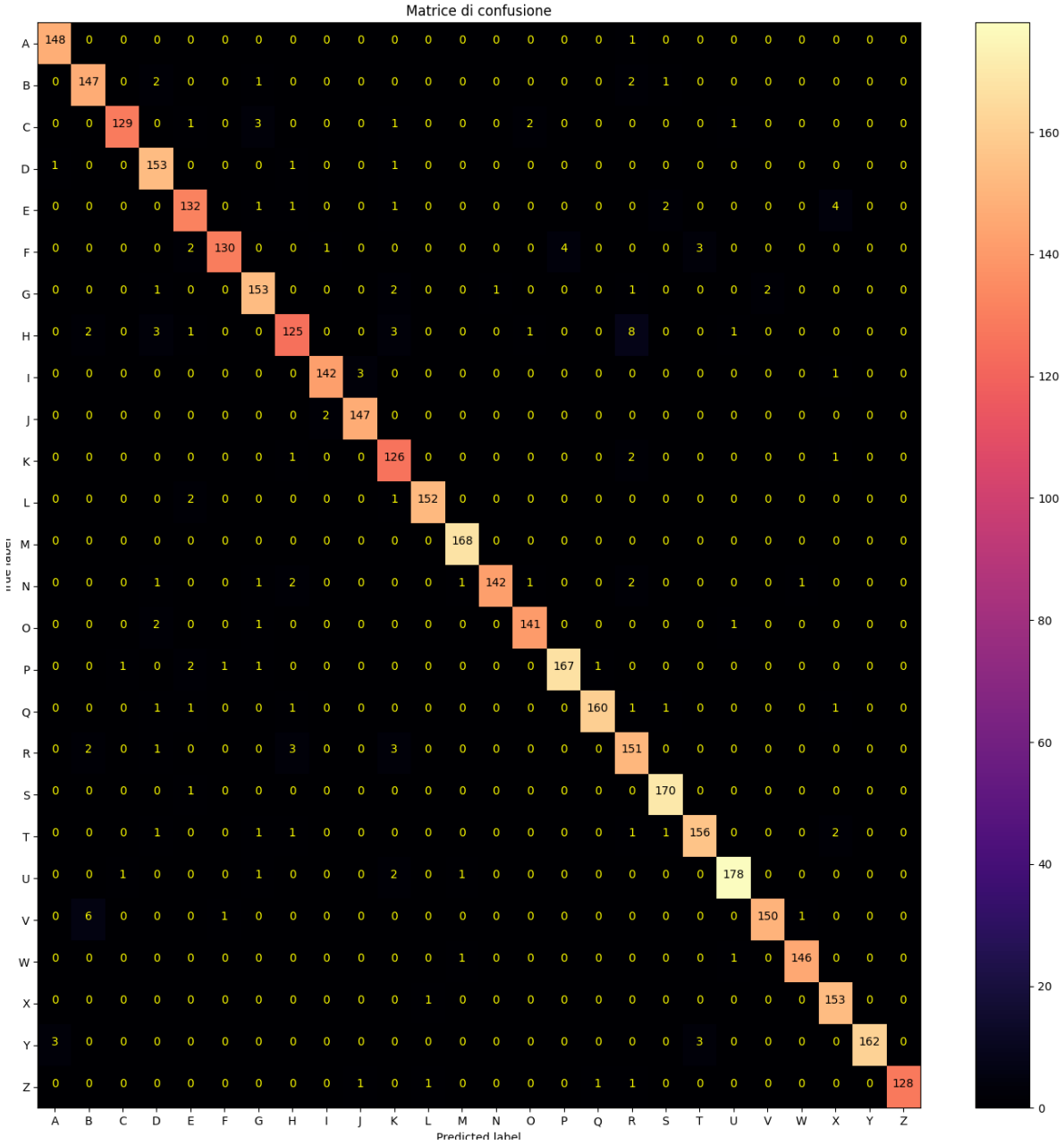
Data la numerosità delle classi, abbiamo optato per una visualizzazione unificata, plottando tutte le curve in un singolo asse cartesiano, rendendo comunque l'idea senza perdita di generalità. Per la rappresentazione visiva dell'intero grafico abbiamo optato per fornire una riduzione dello stesso per favorire la visualizzazione, il range ritenuto maggiormente significativo è stato (0,8;1) per le ascisse (TPR) e (0;0,2) per le ordinate (FPR).

Gli indici di performance che sono stati presi in considerazione sono:

- Precision: $\frac{TP}{TP + FP}$
- Recall: $\frac{TP}{TP+FN}$
- F-score: $2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$

5.1 Reti Neurali

Con il **95%** di accuracy, riusciamo a classificare correttamente 3856 istanze del test set su 4000, ottenendo perciò un modello che riesce a generalizzare su dati mai visti prima, affermazione avvalorata dalla matrice di confusione.

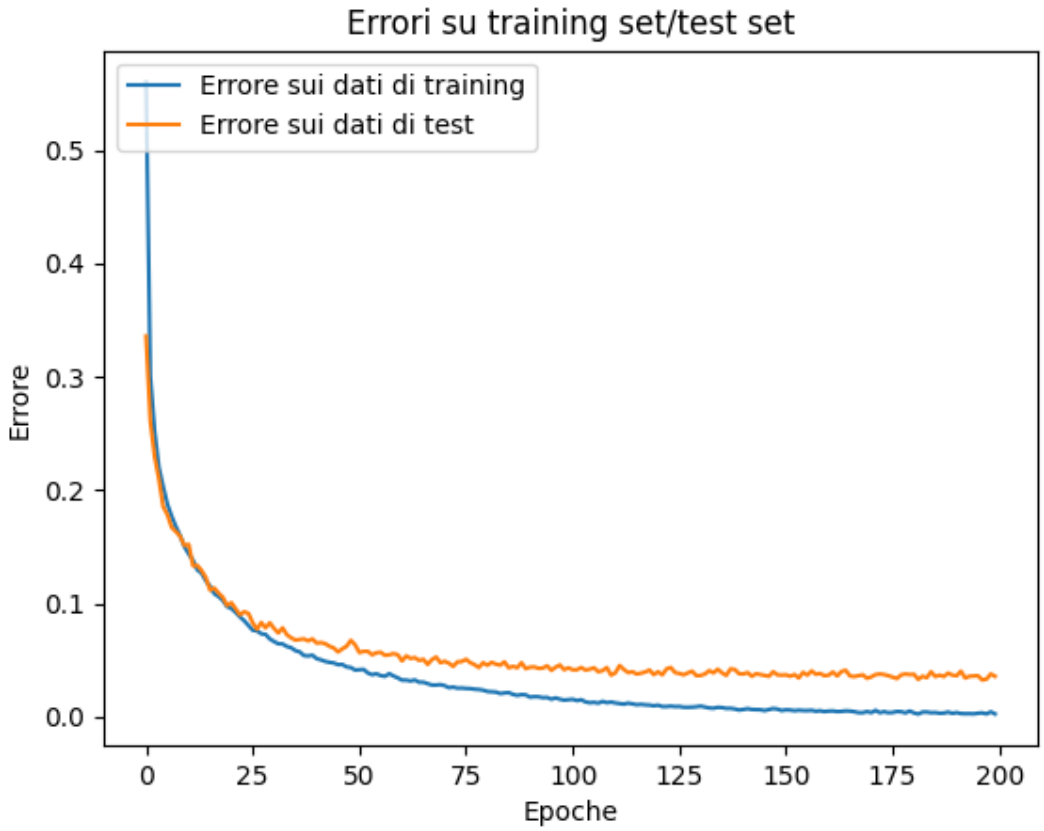


È interessante notare che le due classi che generano maggior incertezza per il modello sono le classi corrispondenti alle lettere R (label predetta) e H (ground truth), tipograficamente molto simili tra loro.

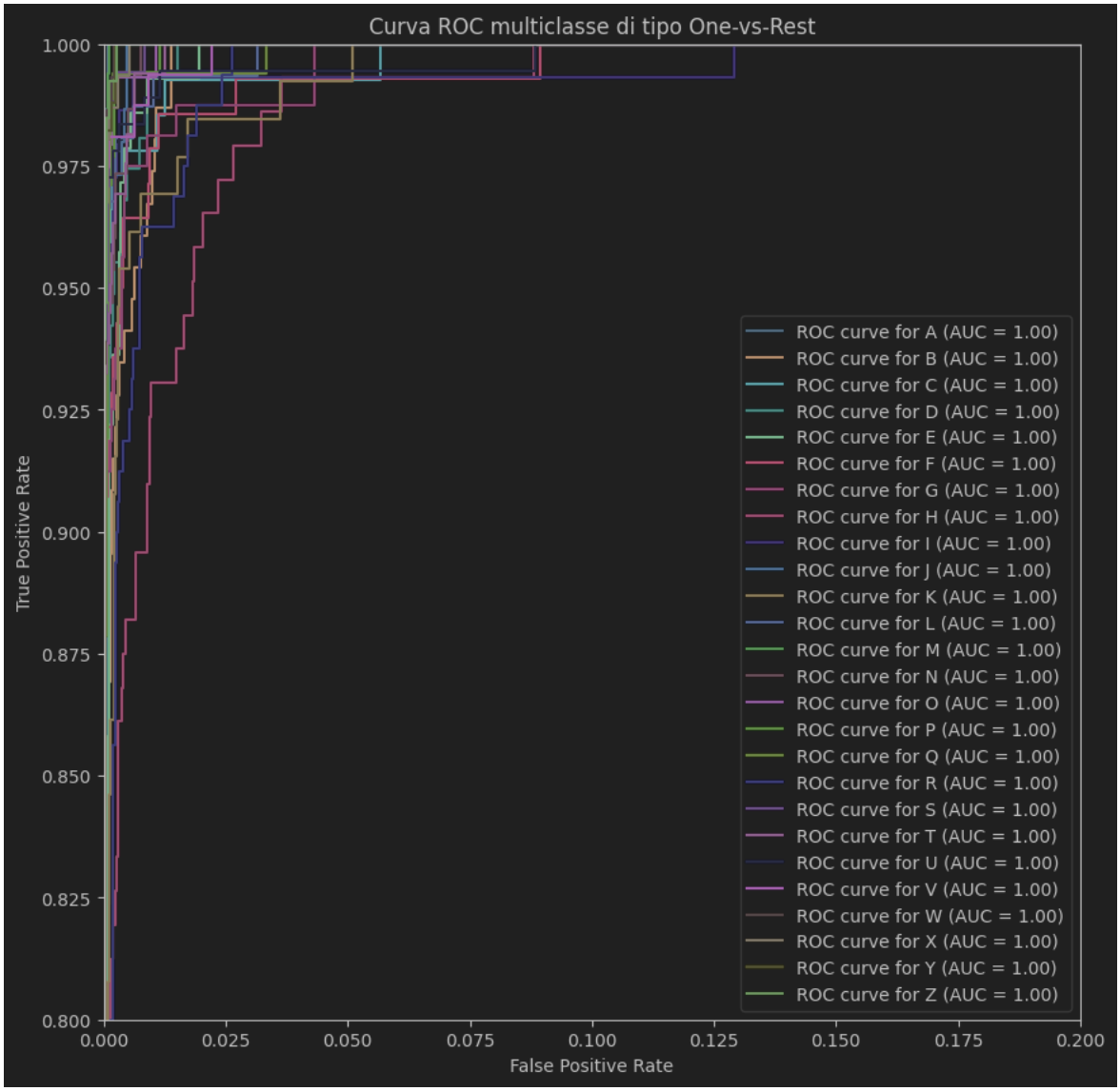
Di seguito sono riportati i valori di queste ottenute dall'ANN.

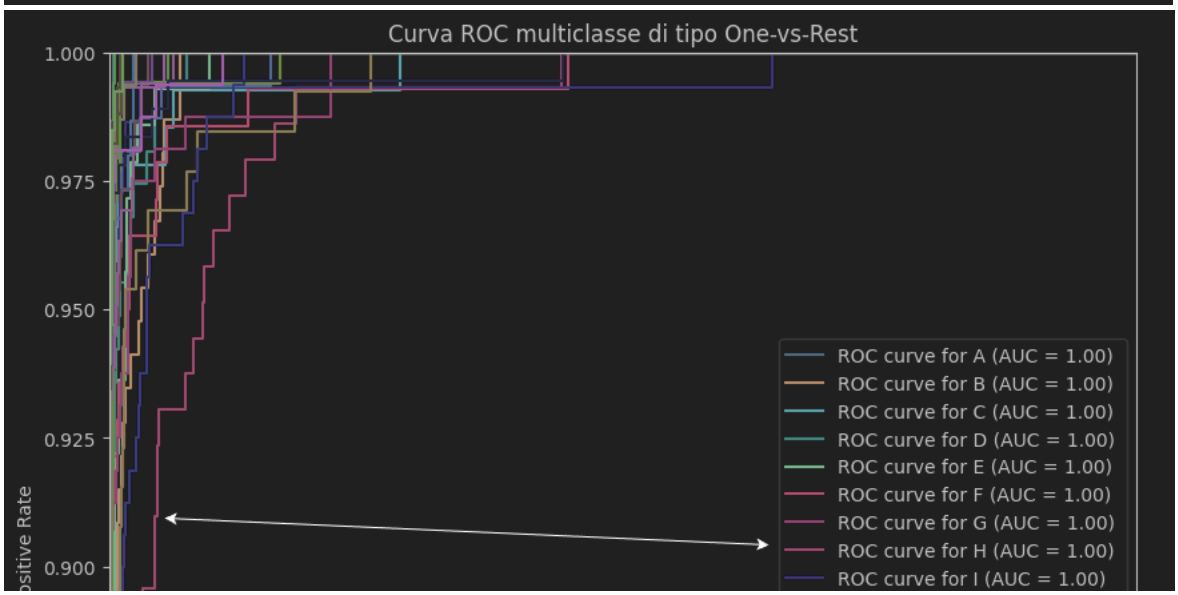
Precision	0.9588
Recall	0.9585
F-score	0,9584

A riprova del fatto che il modello non si trova né in uno stato di overfitting né in uno di underfitting, proponiamo il seguente grafico riportante gli errori compiuti durante le epoche sia del training set sia del test set.



Curva ROC Rete Neurale

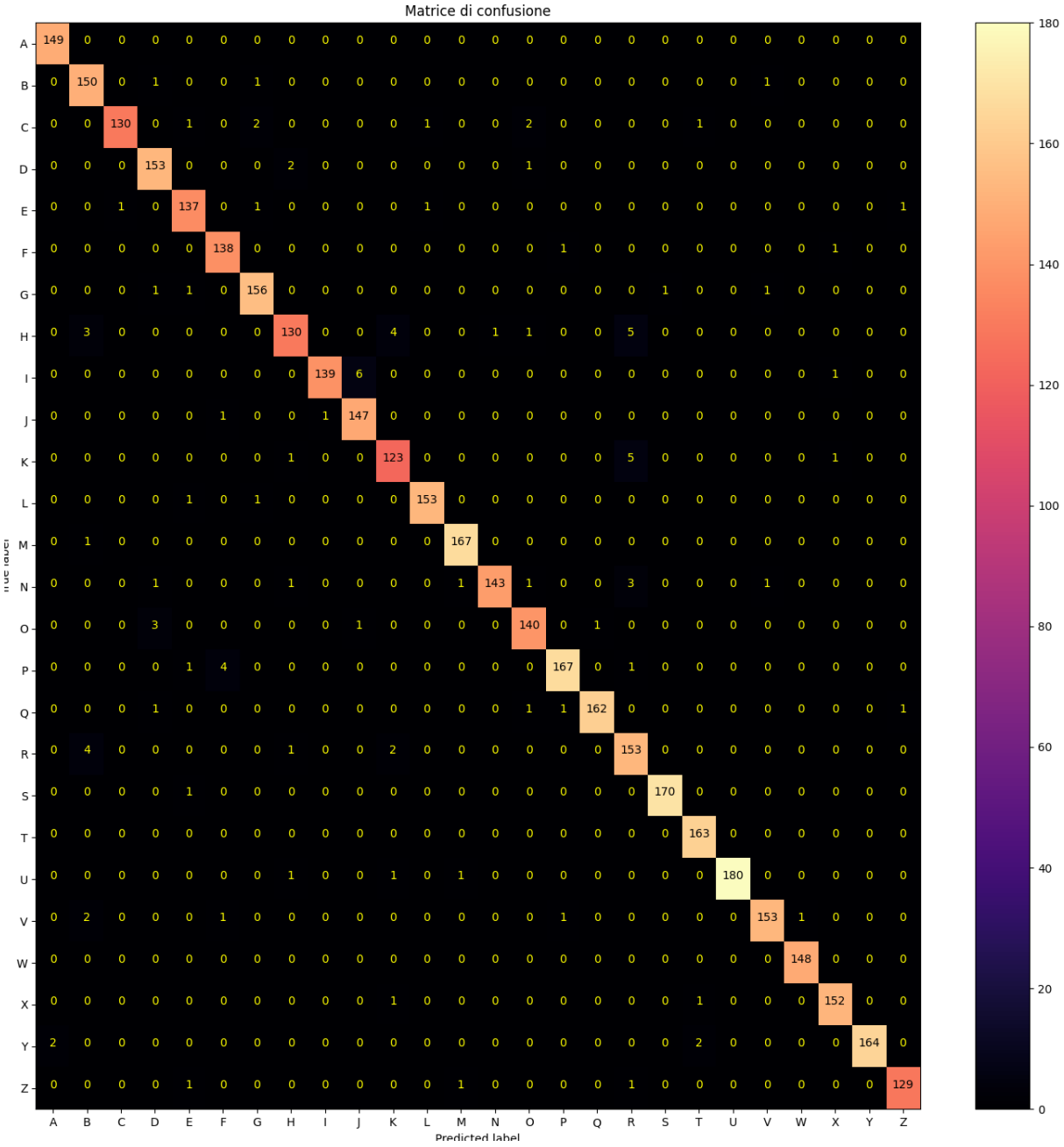




Si noti come la classe più problematica per il modello (quella relativa alla lettera H) abbia un'area di poco inferiore rispetto alle altre. La libreria sklearn in ogni caso è insensibile rispetto a questa irrilevante discrepanza.

5.2 SVM

Con un'accuratezza del **97%**, la Support Vector Machine supera, anche se di poco, le prestazioni della Rete Neurale, classificando 3896 istanze su 4000 nel test set.
Come per le reti neurali, abbiamo prodotto la matrice di confusione:

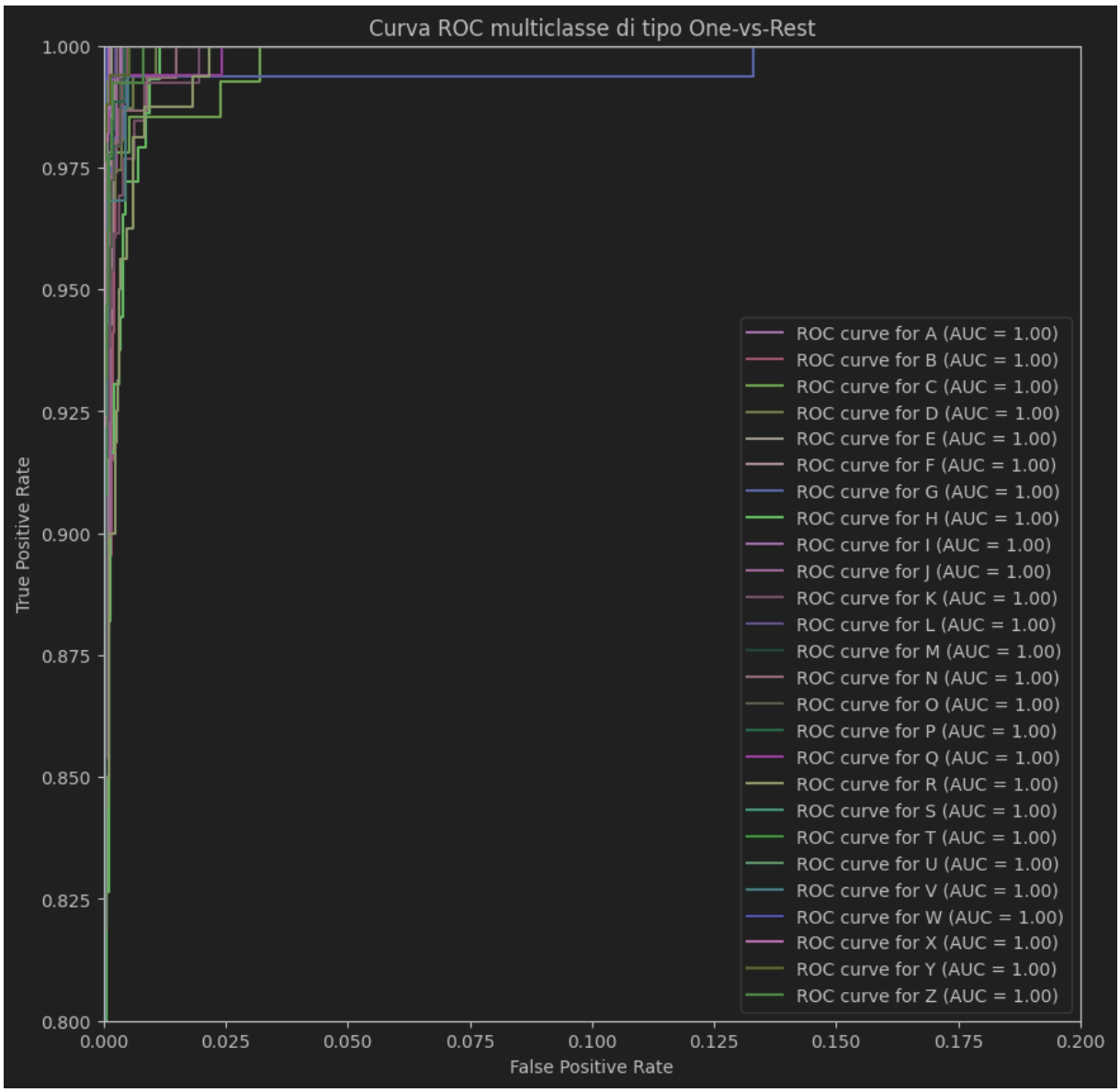


Oltre ai già noti errori dovuti alle somiglianze tra lettere H e R, è possibile notare una ragionevole misinterpretazione tra le lettere I e J, anch'esse tipograficamente simili.

Anche per questo modello, sono state prodotte le metriche di valutazione:

Precision	0.9744
Recall	0.9740
F-score	0,9740

Curva ROC SVM



6. Conclusioni

L'obiettivo primario di questo progetto, ossia quello di indurre due modelli di classificazione supervisionati che non andassero in overfitting e che riuscissero a classificare correttamente dati mai osservati, **è stato raggiunto con successo.**

L'analisi e il confronto dei due modelli, ci ha permesso di stabilire che il modello con l'accuratezza di classificazione maggiore è la **Support Vector Machine (97%).**

Paragonandoli in termini computazionali, l'addestramento della Rete Neurale vince il confronto, poichè per stimare i parametri ottimali della SVM tra quelli da noi scelti, e contemporaneamente eseguire un metodo di valutazione (Stratified K-Fold), è stata necessaria **più di un'ora (3877 secondi),** a causa della già nota complessità computazionale della Grid Search. La rete neurale infatti impiega, per la fase di addestramento senza validazione, un tempo di **60 secondi**; la fase di validazione, al termine della quale produciamo un intervallo di confidenza, consiste in 5 addestramenti indipendenti: ci si aspetta un tempo di 5 minuti, ma l'opzione di Early Stopping riduce i tempi a **3 minuti e 43 secondi**.

Possiamo quindi concludere che nessuno dei due modelli sia del tutto superiore all'altro, se sia ha necessità di massimizzare le performance (accuracy, precision, recall, f-score) senza preoccuparsi del tempo di addestramento allora riterrò migliore la SVM; invece se ho bisogno di un addestramento rapido a scanso di ridurre leggermente l'accuratezza sceglierò la rete neurale.

Infine abbiamo tentato di produrre dati artificialmente per poter fare inferenza, tuttavia, data la natura del dataset e la scarsa documentazione annessa, non ci è stato permesso di farlo. Essendo ogni istanza caratterizzata da 16 valori scalati tra 0 e 15, non possiamo, senza conoscere la scala, ricondurci ai dati originali prima dello scaling (ossia i veri valori, caratterizzati da una propria precisione, dei vari parametri). Quindi pur conoscendo la semantica del dataset, ossia cosa ogni feature rappresenta, risulta difficile produrre dei valori numerici che seguano la distribuzione delle istanze di esempio a partire da immagini di lettere scritte a mano da terzi.